C.E.S.A.R school

Análise e Visualização de Dados

Streamlit

parte 2

Agenda

- Atualizações do repositório
- Gráficos interativos e cache
- Dashboard 1 Completo
- Teoria de métricas básicas
- Identidade visual
- Dashboard 2: KPIs e métricas
- Hands-on: construção dos dois dashboards

Atualizações do repositório

- Nova pasta **entregas/**
  - Objetivo: rascunhos e justificativas
  - Independente de prompts/
- A pasta **prompts/**
  - Guarda só o que é relacionado ao uso de IA
  - Templates de prompts
  - Declaração de uso de IA
- Arquivo novo: **ROTEIRO.md**
  - Passo a passo de todas as atividades
- Observação: quem deu fork ou clonou antes de mudanças:
  - git pull do upstream para puxar as atualizações
  - ajustar manualmente o diretório

Gráficos Interativos no Streamlit

- `st.pyplot` para usar **Matplotlib ou Seaborn**
  - reaproveita a lógica do que vimos na primeira unidade
  - imagens **estáticas**
- `st.plotly_chart` para utilizar o **Plotly**
  - gráficos **interativos** com zoom e hover
- `st.altair_chart` ou `st.vega_lite_chart` para utilizar **Altair ou Vega-Lite**
  - permite **seleção cruzada** entre gráficos (como PBI)
    - exige declarar a seleção e concatenar os charts no código
- `st.line_chart` / `st.bar_chart` / `st.scatter_chart` para gráficos nativos
- Observação: `st.bokeh_chart` foi descontinuado!

Por que usar cache? @st.cache_data

- Se não usar cache
  - Todo clique recarrega e reprocessa o dataset inteiro
- Se usar cache
  - O dado só é lido ou processado uma vez
  - Reaproveitamento entre interações

```
@st.cache_data
def carregar_dados():
     return pd.read_csv("data/clean/dataset.csv")
```

Abordagens para Dashboards

- Para o **dashboard 1:**
  - Abordagem mínima é reaproveitar as visualizações já feitas na EDA
  - Se usou seaborn ou matplotlib, precisa usar st.pyplot
- Para o **dashboard 2:**
  - Desejável utilizar Altair
  - Aproveitar a interatividade
  - Seleção cruzada entre gráficos

Objetivo para o Dashboard 1

Filtro muda
(seletor de variáveis / slider de intervalo)

↓

Recalcula matriz de correlação

↓

Recalcula distribuições

↓

Heatmap e Histograma atualizam na tela

Observação

- Relembrando que correlação só é válido para variáveis quantitativas
  - Se o seu dataset da EDA era majoritariamente categórico
  - Pode trocar dataset!
  - Mas para isso precisa fazer uma EDA mínima de base

Métrica

- **Clara**
  - Entendível, mesmo que não precisa de explicação extra
- **Acionável**
  - Alguém pode decidir algo sobre isso
- **Ligada a um público**
  - Comunica algo para alguém
- **Tipos comuns**
  - Contagens
  - Médias
  - Proporções
  - Taxa
  - Variações

Identidade visual

- Relembrando a parte de organização visual
  - Precisamos de uma unidade visual
- Para os gráficos e visualizações
  - https://www.learnui.design/tools/data-color-picker.html#palette
  - https://projects.susielu.com/viz-palette
  - coolors.co
- Para a UI como um todo
  - realtimecolors.com
    - Para pré-visualizar texto, fundo, cores primárias e secundárias, etc
  - fonts.google.com
    - Para seleção de fontes

Identidade visual - .streamlit/config.toml

Chave	O que controla
`primaryColor`	cor de destaque (botões, sliders ativos)
`backgroundColor`	fundo principal
`secondaryBackgroundColor`	fundo de sidebar/widgets
`font`	fonte da aplicação

- Exercício rápido: escolher 2-3 cores coerentes com o tema do próprio dataset/dashboard

Identidade Visual no Streamlit

.streamlit > config.toml

```toml
# Configuração do Streamlit. Descomente e ajuste conforme o projeto avançar.

[theme]
paddingTop = "0.0rem"     # espaço abaixo da barra do topo (valor entre aspas)
# font = "sans serif"     # "sans serif", "serif" ou "monospace"

[theme.light]
primaryColor = "#c1121f"            # cor de destaque (widgets, links): mesmo vermelho dos histogramas
backgroundColor = "#ffffff"         # fundo da página
secondaryBackgroundColor = "#f0f2f6"  # fundo da sidebar e de widgets
textColor = "#31333f"               # cor do texto
borderColor = "#d5dae5"

[theme.dark]
primaryColor = "#c1121f"
backgroundColor = "#0e1117"
secondaryBackgroundColor = "#262730"
textColor = "#fafafa"
borderColor = "#41444e"
```

Exibição de Indicadores

- Utilizamos a função st.metric para os cartões de KPI também
- Convencionalmente adotamos entre 3 a 5 KPIs por dashboard
  - Menos do que 3 dá uma sensação de "vazio"
  - Mais do que 5 fica difícil de compreender
- Também podemos trazer visualizações simplificadas, como discretizações de uma variável

Como planejar métricas

- Para cada métrica, planeje:
  - Variável:
  - O que mede:
  - Por que foi escolhida:
  - Público-alvo:
  - Qual decisão pode ser apoiada por essa métrica?
  - O que comunica ao público-alvo:

- Devem ser sugeridas entre 3 a 5 métricas
- Essas informações vão ser entregues em `entregas/justificativa-metricas.md`

Como planejar métricas

- Antes de criar visualizações:
  - Selecionar variável de interesse
  - Entender o tipo de métrica a usar
  - Fazer códigos (em notebook) para testar métricas e cálculos
  - Fazer plots prévios (em notebook)
- Após planejar, no app .py vai ser utilizado o streamlit para calcular a métrica

```
media_idade =
df.groupby("sexo")["idade"].mean()
delta =
df["colesterol"].pct_change().mean()
```

Bloco 1 (pandas): bom pra explorar/validar a métrica em notebook, antes de fechar o cálculo

```
st.metric("Idade média",
f"{media_idade:.1f}")
st.metric("Variação colesterol",
f"{delta:.1%}")
```

Bloco 2 (st.metric): entra na aplicação — recalcula sozinho quando o dataset/filtro muda, confirmando a métrica ao vivo

Hands-on: o que construir hoje

- **Atualizar o repositório** com as mudanças apresentadas
- **Alinhar dashboard 1 até os widgets** (retomando o esqueleto da Aula 21)
- Definir e aplicar **identidade visual** da aplicação
- **Selecionar visuais** de correlação + distribuição e completar o **Dashboard 1**
  - **Testar:**
    - 0 variáveis, 1 variável, 2+ variáveis, combinações distintas de widgets
- Estruturar **métricas e KPIs** para o **Dashboard 2**
  - seleção de variáveis → proposição de métricas → elaboração de visualizações
- Rascunhar o Dashboard 2 e criar as tabs para os dois dashboards
- Agrupar 1 variável e mostrar um visual do tipo gráfico de barra

Hands-on: entrega no Git

- **Dashboard 1 completo** e testado
- Rascunho do Dashboard 2 em **entregas/rascunho-dashboard2.jpg**
- Declaração de uso de IA atualizada (**prompts/declaracao-uso-ia.md**)
- Justificativa das métricas (**entregas/justificativa-metricas.md**)
- Não precisa fechar todos os objetivos do Dashboard 2 hoje
- **Commit e push**
  - Ao menos um commit ao final da aula, com:
    - Dashboard 1 completo e testado
    - Rascunho do Dashboard 2
    - Justificativa das métricas
  - Essa é a entrega do dia
  - A avaliação será feita com base na entrega final, porém será observado o histórico das entregas



C.E.S.A.R school

MAIRA LIRA

mfal@cesar.school

in/maira-lira